CHANDIGARH UNIVERSITY

Discover. Learn. Empower.

Gharuan, Mohali, Punjab

COMPUTER ORGANIZATION AND ARCHITECTURE

Assignment: Case Study No. 3

CACHE MEMORY PERFORMANCE ANALYSIS

Student Name	Hriday Katariaa
UID / Roll Number	25BAI10046
Course	Bachelor of Technology (B.Tech)
Branch	CSE AI (Microsoft)
Academic Year	2026–27
Semester	3rd Semester
Section	601-A
College / University	Chandigarh University
Subject	Computer Organization and Architecture
Subject Code	25CST-238
Submitted To	Mrs. Ruchika Gupta Mam

1. Introduction

Memory is the part of a computer that stores programs and data. The CPU keeps asking memory for instructions and operands, so the speed of memory directly affects the speed of the whole system. A modern CPU can work in a fraction of a nanosecond, but main memory (RAM) is much slower. If the CPU had to wait for RAM on every access, it would spend most of its time idle.

Cache memory solves this problem. It is a small and very fast memory that keeps copies of the data and instructions the CPU is likely to need soon. It is placed between the CPU and RAM so that most requests can be answered quickly from the cache, and only a few have to go to the slower RAM. Because of this, the average time taken to read data falls sharply and the processor performs better.

2. Scenario and Problem Statement

A processor makes 1,000 memory accesses. Of these, 850 are found in the cache (hits) and 150 are not (misses). The values given in the assignment are shown below.

Parameter	Value	Unit
Total memory accesses	1,000	accesses
Cache hits	850	accesses
Cache misses	150	accesses
Cache hit time	2	ns
Additional miss penalty	50	ns

Problem statement in my own words: We have to find out how well the cache is working by calculating its hit ratio and miss ratio. Then we have to find the average time needed for one memory access (AMAT) and show how this time depends on the hit ratio.

3. Objectives of the Case Study

- To understand what cache memory is and where it sits in the memory hierarchy.
- To distinguish between cache hits and cache misses.
- To calculate the cache hit ratio and miss ratio from the given data.
- To calculate Average Memory Access Time (AMAT) using the given formula.
- To understand locality of reference and the link between hit ratio and AMAT.

4. Answers to the Case Study Questions

Q11. Define Cache Memory.

Definition: Cache memory is a small, high-speed memory that stores copies of frequently used data and instructions from main memory so that the CPU can get them quickly.

Purpose: Its job is to reduce the speed gap between the fast CPU and the slow RAM, so the CPU waits less for data.

Location: In the memory hierarchy, the order from fastest to slowest is registers, cache, main memory, and secondary storage. Cache sits just below the registers, between the CPU and RAM. Many processors have more than one level (L1, L2, L3), with L1 being the smallest and fastest.

Importance: Cache is costly and small, but because programs reuse data, even a small cache answers most requests. This gives performance close to a fast memory at the cost of a much cheaper one.

Q12. Distinguish Between Cache Hits and Cache Misses.

A **cache hit** happens when the data the CPU asks for is already present in the cache. A **cache miss** happens when it is not present, so the data must be brought from RAM.

Point	Cache Hit	Cache Miss
Meaning	Requested data is found in the cache.	Requested data is not in the cache.
What happens	Data goes to the CPU straight from the cache.	The block is fetched from RAM, copied into the cache, and then given to the CPU.
Performance impact	Fast. Takes only the hit time (2 ns here).	Slow. Takes the hit time plus the miss penalty (2 + 50 = 52 ns here).
Simple example	Reading the same array element again inside a loop.	Reading a variable for the very first time.

Q13. Calculate Cache Hit Ratio.

The hit ratio is the fraction of memory accesses that are found in the cache.

$$\text{Hit Ratio} = \text{Number of Cache Hits} \div \text{Total Memory Accesses}$$

Given: Cache hits = 850 and Total accesses = 1,000.

$$\text{Hit Ratio} = 850 \div 1000$$

$$\text{Hit Ratio} = 0.85$$

Result: Hit ratio = **0.85**, or **85%**.

Q14. Calculate Cache Miss Ratio.

The miss ratio is the fraction of accesses that are not found in the cache.

$$\text{Miss Ratio} = \text{Number of Cache Misses} \div \text{Total Memory Accesses}$$

Given: Cache misses = 150 and Total accesses = 1,000.

$$\text{Miss Ratio} = 150 \div 1000$$

$$\text{Miss Ratio} = 0.15$$

Result: Miss ratio = **0.15**, or **15%**.

Relationship: Every access is either a hit or a miss, so the two ratios always add up to 1. Here, 0.85 + 0.15 = 1. Therefore, Miss Ratio = 1 − Hit Ratio, and Hit Ratio = 1 − Miss Ratio.

Q15. Why Is Locality of Reference Important?

Locality of reference means that programs tend to access the same data, or data close to it, again and again over a short time. A cache works well only because of this behaviour. There are two types.

- **Temporal locality:** Data used recently is likely to be used again soon. Example: a loop counter, or a variable inside a loop, is read in every iteration.
- **Spatial locality:** Data near a recently used address is likely to be used soon. Example: when a program reads an array one element after another, the next elements are stored next to the current one.

Because of temporal locality, the cache keeps recently used data. Because of spatial locality, a cache copies a whole block of nearby data on one miss, so the next accesses become hits. Good locality therefore gives a high hit ratio and a lower AMAT. Without locality, the cache would miss often and would help very little.

5. Average Memory Access Time (AMAT)

AMAT is the average time the CPU needs for one memory access, counting both hits and misses. The formula given in the assignment is:

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

Given: Hit time = 2 ns, Miss rate = 0.15 (from Q14), Miss penalty = 50 ns.

Step 1: Write the formula.

$$\text{AMAT} = \text{Hit Time} + (\text{Miss Rate} \times \text{Miss Penalty})$$

Step 2: Substitute the values.

$$\text{AMAT} = 2 \text{ ns} + (0.15 \times 50 \text{ ns})$$

Step 3: Multiply first. The miss rate tells us that on average each access carries 15% of the 50 ns penalty.

$$0.15 \times 50 \text{ ns} = 7.5 \text{ ns}$$

Step 4: Add the hit time.

$$\text{AMAT} = 2 \text{ ns} + 7.5 \text{ ns} = 9.5 \text{ ns}$$

Final result: AMAT = **9.5 ns**.

Difference between the three terms:

- **Hit time (2 ns):** the time to access the cache. It is spent on every access, whether hit or miss.
- **Additional miss penalty (50 ns):** the extra time spent only on a miss to fetch data from RAM. It is added on top of the hit time.
- **AMAT (9.5 ns):** the overall average per access. It is the hit time plus the penalty spread over all accesses (7.5 ns on average).

6. CPU–Cache–RAM Diagram

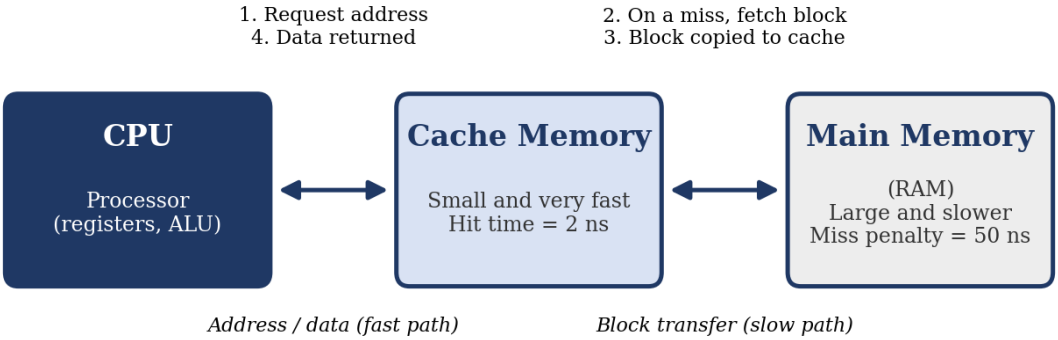


Figure 1: CPU – Cache – RAM organisation

The CPU sends every memory request to the cache first. If the data is there (a hit), it returns to the CPU quickly. If not (a miss), the cache asks RAM for the block, stores a copy, and then passes the data to the CPU. The double-headed arrows show that information moves in both directions. The CPU does not talk to RAM directly in this model.

7. Three Important Observations

1. **The cache is working effectively.** A hit ratio of 0.85 means 85 out of every 100 accesses are served by the cache in only 2 ns. Only a small share of requests have to go to the slow RAM.
2. **Misses raise the average access time.** A hit costs 2 ns, but a miss costs 52 ns. The 15% misses add 7.5 ns to the average, so AMAT becomes 9.5 ns instead of 2 ns. Although misses are fewer in number, each one is very costly.
3. **Better locality improves performance.** Programs with good temporal and spatial locality get more hits, which reduces the miss rate and lowers AMAT. So writing programs that reuse data, and designing caches that exploit this, helps the CPU run faster.

8. Relationship Between Hit Ratio and Average Access Time

Hit ratio and AMAT move in opposite directions. Since Miss Rate = 1 − Hit Ratio, the AMAT formula can be written as AMAT = Hit Time + (1 − Hit Ratio) × Miss Penalty. A higher hit ratio makes the term (1 − Hit Ratio) smaller, so less penalty is added and AMAT goes down. A higher miss ratio does the opposite: more accesses pay the 50 ns penalty, so AMAT goes up.

To see this, the table below keeps the hit time (2 ns) and penalty (50 ns) the same and changes only the hit ratio. The 85% row is our case study. The other rows are just illustrative values calculated with the same formula.

Hit Ratio	Miss Ratio	Penalty added (ns)	AMAT (ns)
70% (0.70)	0.30	0.30 × 50 = 15.0	2 + 15.0 = 17.0
85% (0.85) – given	**0.15**	**0.15 × 50 = 7.5**	**2 + 7.5 = 9.5**
95% (0.95)	0.05	0.05 × 50 = 2.5	2 + 2.5 = 4.5

The table shows that when the hit ratio rises from 70% to 95%, AMAT falls from 17.0 ns to 4.5 ns. This is why designers try to raise the hit ratio. In the given scenario, the 85% hit ratio gives an AMAT of 9.5 ns, which is much closer to the cache speed (2 ns) than to the RAM-miss cost (52 ns).

9. Conclusion

This case study showed how to judge the performance of a cache using simple calculations. From 850 hits out of 1,000 accesses, the hit ratio is 0.85 (85%) and the miss ratio is 0.15 (15%). Using the formula AMAT = Hit Time + (Miss Rate × Miss Penalty), the average memory access time comes to 2 + (0.15 × 50) = 9.5 ns.

The result shows why cache memory is so important. It is small, yet it lets the CPU avoid the slow RAM in most accesses, because programs follow temporal and spatial locality. Even a small number of misses affects the average time, since each miss is expensive. Therefore, the hit and miss ratios are the key measures of cache quality. A higher hit ratio means a lower AMAT, less waiting for the CPU, and better overall processor performance.

10. References

[1] Hennessy, J. L. and Patterson, D. A., Computer Architecture: A Quantitative Approach, Morgan Kaufmann.

[2] Patterson, D. A. and Hennessy, J. L., Computer Organization and Design: The Hardware/Software Interface, Morgan Kaufmann.

[3] Stallings, W., Computer Organization and Architecture: Designing for Performance, Pearson.

[4] Mano, M. M., Computer System Architecture, Pearson.

Note: Editions and years are not listed. Please add them from the copy you actually refer to. The problem data are taken from the assignment PDF (Case Study 3).